Facultad de Ciencias de la Administración
UNER

TRABAJO FINAL INTEGRADOR

Programación IV – 2026 2do cuatrimestre

Licenciatura en Sistemas

OBJETIVOS

El Trabajo Final Integrador tiene como objetivos que el estudiante:

- Defina la estructura de los documentos web y las relaciones entre estos.
- Establezca estilos apropiados para dispositivos de diferentes resoluciones.
- Diseñe e implemente servicios web basados en arquitecturas REST.
- Utilice estructuras de datos apropiadas para la gestión eficiente de la información.

CONDICIONES DE ENTREGA

- El Trabajo Final Integrador deberá ser:
  - Realizado en forma grupal.
  - Los grupos deberán contar con un mínimo de 3 y un máximo de 5 alumnos.
  - Cargado en la sección del Campus Virtual como un archivo de texto plano (txt) que cuente con la URL al repositorio GitHub. Además, para asegurar el acceso del equipo docente, se sugiere cargar un archivo comprimido en formato ZIP, RAR, TAR.GZ u otro formato similar. con el proyecto completo excluyendo las dependencias (node_modules).
  - Indicar número de grupo, apellido y nombre de los integrantes.
  - Entregado antes de la fecha límite informada en el campus.
- Las soluciones deben ser de autoría propia. Aquellas que se detecten como idénticas entre diferentes grupos serán desaprobadas para todos los involucrados.
- Se valorarán la exactitud, eficiencia y prolijidad (identación y otras buenas prácticas) de las soluciones planteadas, así como también la calidad de la exposición realizada.
- Los integrantes del equipo deberán realizar una exposición del proyecto, detallando su funcionamiento y las decisiones de diseño tomadas.

ENUNCIADO

El área de sistemas del municipio local decidió implementar un sistemas de registros de incidencias. A tal fin le encarga el desarrollo del proyecto a su equipo.
El sistema deberá contar con una aplicación web disponible en la intranet de la institución que permita llevar a cabo las siguientes tareas:

Requisitos Funcionales

Login de usuario o pantalla de inicio de sesión para todos los roles de usuario.

Empleados Municipales:

- Listar los artículos.
- Crear una incidencia.
- Listar las incidencias creadas por el empleado.
- Cancelar una incidencia propia con estado “pendiente”.

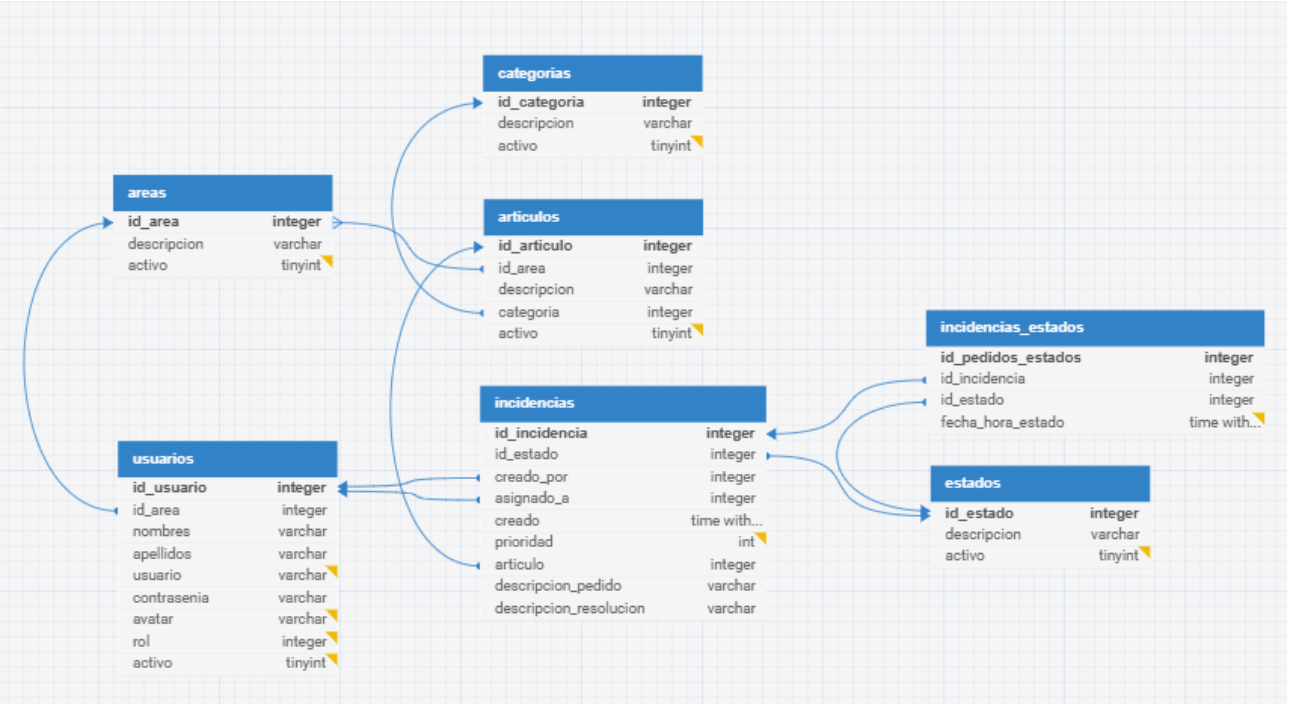
Empleado área de Sistemas:

- Listar las incidencias asignadas a su cargo.
- Finalizar incidencias.
- BREAD completo de Categorías.
- BREAD completo de Artículos.

Director del área de Sistemas:

- Panel de resumen o dashboard (Totales de incidencias según estados, Totales de incidencias por fecha, Incidencias Prioritarias).
- Generar reportes estadísticos de incidencias.
- Asignar incidencias a empleados del área de Sistemas.
- Cancelar una incidencia con estado “pendiente”.

El diagrama de base de datos a respetar:



Requerimientos técnicos

- Aplicación con interfaz web utilizando diseño responsivo.
- Autenticación de usuarios utilizando tokens JWT.
- API Rest cumpliendo con las buenas prácticas de diseño de APIs REST.
- Generación de reportes en PDF.
- Envío de correo electrónico una vez que una incidencia sea finalizada o cancelada.
- Gestionar información sensible del servicio mediante variables de entorno.
- No utilizar borrados físicos sino “soft deletes”, es decir utilizar los campos de estado pensados para indicar que los registros no están activos.
- Documentación del API haciendo uso de Swagger.
- Utilización de middlewares:
    - Validaciones. Ej.: express-validator.
    - Registro de solicitudes. Ej.: Morgan.
    - Carga de archivos. Ej.: Multer.

Opcionales:

- Pantalla de cambio de contraseña de usuario.
- BREAD de usuarios.
- BREAD de áreas.
- Incluir funcionalidades SSE o WebSocket que permitan saber a los administradores en tiempo real la información actualizada sobre las incidencias.

Notas:

- Un empleado pertenece sólo a un área.
- Las incidencias se crean para un artículo.
- Buenas prácticas:
    - Versionado del API Rest.
    - División en capas.
    - Nombres correctos en rutas.
    - Validación / Transformación de datos de entrada.
    - Uso adecuado de códigos de estados.
    - Respuestas consistentes.
- **BREAD** es el acrónimo de:
    - **B**rowse: Mostrar todos los recursos.
    - **R**ead: Leer / Mostrar un único recurso.
    - **E**dit: Actualizar / Modificar un recurso.
    - **A**dd: Agregar / Crear un nuevo recurso.
    - **D**elete: Eliminar / Borrar un recurso.

Rúbricas de Evaluación

Escala por criterio:

0 = No cumple / incorrecto
1 = Parcial / incompleto / con errores
2 = Cumple correctamente y de forma consistente

Nota = Puntaje total / 10

1. CRUD de entidades

0: Faltan operaciones básicas o no funcionan.
1: CRUD incompleto o con errores de persistencia.
2: CRUD completo y funcional para las entidades requeridas.

2. Requisitos Funcionales (Listar los artículos, Panel de resumen o dashboard, Finalizar incidencias, etc.)

0: No implementadas o incorrectas.
1: Parcialmente implementadas o inconsistentes.
2: Correctas y coherentes en toda la API.

3. Endpoints y diseño REST

0: Rutas incorrectas o no REST.
1: REST parcial o inconsistente.
2: Diseño REST claro (recursos, verbos, códigos HTTP).

4. Persistencia.

0: Errores de guardado o inconsistencias.
1: Persistencia funcional sin manejo de casos críticos.
2: Persistencia correcta con manejo adecuado de transacciones.

5. Requerimientos técnicos (express-validator - JWT - Swagger, etc.)

0: No implementadas.
1: Implementadas parcialmente o con errores.
2: Implementadas correctamente y funcionales.